

DATA STRUCTURES **IN** **PYTHON**

PRESENTATION BY

LEVIATHAN 7



AGENDA

Python Data Structures — From Basics to Practical Use

01 LIST

 Sanjay

02 TUPLE

 Vamshi

03 SETS

 Parves

04 DICTIONARIES

 Mones
h

05 METHODS & OPERATIONS

 Tej
a

06 NESTED STRUCTURES

 Malar

07 ITERATION & MANIPULATION

 Asmith
a

LISTS IN PYTHON

Ordered, mutable collections



List Memory Layout

```
numbers = [10, 20, 30, 40]
```

INDEX

0

1

2

3



VALUE

10

20

30

40

Access by Index: numbers[2]

→ 30



ORDERED

Elements maintain their insertion order and can be accessed by zero-based index.



MUTABLE

A list can be changed after creation: elements can be added, removed, or replaced.



DUPLICATES ALLOWED

Repeated values are allowed and stored as distinct indexed items.



HETEROGENEOUS

A single list can store different data types (strings, integers, floats, booleans).



HOW A LIST IS CREATED

1. BASIC SYNTAX

```
items = [1, 2, 3]
```

2. EMPTY LIST

```
items = []
```

3. LIST() CONSTRUCTOR

```
list((1, 2, 3))
```

→ [1, 2, 3]

INDEXING & SLICING



01 INDEXING

```
fruits = ["Apple", "Banana", "Cherry", "Mango"]
```

+ INDEX

0

1

2

3

VALUE

"Apple"

"Banana"

"Cherry"

"Mango"

- INDEX

-4

-3

-2

-1

FIRST ELEMENT

```
fruits[0]
```

→ "Apple"

LAST ELEMENT

```
fruits[-1]
```

→ "Mango"



Positive starts at 0 (left-to-right); Negative starts at -1 (right-to-left).



02 SLICING

```
numbers = [10, 20, 30, 40, 50]
```

```
numbers[1:4]
```

→ [20, 30, 40]

10

20

30

40

50

[START : STOP : STEP]

START: 1
(incl.)

STOP: 4
(excl.)

STEP:
1

REVERSAL SLICING

```
numbers[::-1]
```

→ [50, 40, 30, 20, 10]



Slicing always creates a new list containing the selected elements.

LISTS CAN CHANGE

BEFORE numbers = [10, 20, 30]



REASSIGN INDEX 1
numbers[1] = 200



AFTER [10, 200, 30]

Element Successfully Changed

append(x)

END

Adds a single element to the end of the list.

```
[1, 2].append(3)
```

```
[1, 2, 3]
```

insert(i, x)

INDEX

Inserts element at the specified index position.

```
[1, 3].insert(1, 2)
```

```
[1, 2, 3]
```

extend(iter)

BULK

Appends all elements from an iterable to the end.

```
[1, 2].extend([3, 4])
```

```
[1, 2, 3, 4]
```

remove(x)

BYVALUE

Removes first matching occurrence of value.

```
[1, 2, 3].remove(2)
```

```
[1, 3]
```

pop([i])

RETURNS

Removes and returns item at index (default: last).

```
[10, 20, 30].pop()
```

30 removed

clear()

RESET

Empties the entire list of all its elements.

```
[1, 2, 3].clear()
```

```
[]
```

WORKING WITH LISTS

Core Operators & Loops



01 CONCATENATION

+ Operator

Merges two distinct lists into a brand new combined sequence.

`[1, 2] + [3, 4]`



`[1, 2, 3, 4]`



02 REPETITION

* Operator

Duplicates the elements of a list a specified integer number of times.

`[1, 2] * 2`



`[1, 2, 1, 2]`



03 MEMBERSHIP

in / not in

Tests whether a value exists within the list, returning a boolean result.

`2 in [1, 2, 3]`

→ **True**

`5 not in [1, 2, 3]`

→ **True**



04 ITERATION

for loop

```
for item in [10, 20, 30]:  
    print(item)
```

10

20

30

`len(nums)` Length

`min(nums)` Minimum

`max(nums)` Maximum

`sum(nums)` Sum total

TUPLES IN PYTHON



Tuple Memory Structure `t = ("A", "B", "C", "D")`

FIXED REFERENCES

INDEX	0	1	2	3
	↓	↓	↓	↓
VALUE	"A"	"B"	"C"	"D"

Positional Access: `t[2]`

→ "C"

Ordered, immutable collections

↓ ORDERED

Elements maintain their order and can be accessed by index.

🔒 IMMUTABLE

Tuple elements cannot be added, removed, or reassigned.

📄 DUPLICATES ALLOWED

Repeated values are permitted and stored distinctly.

🧩 HETEROGENEOUS

A tuple can store different data types together.



How a Tuple is Created

MixedTypes: `("Vamshi", 22, 8.94, True)`

1. BASIC SYNTAX

```
t = ("A", "B", "C")
```

2. SINGLE ITEM TRAILING COMMA

(5,) ✓ **Tuple**

(5) ✗ **Int**



3. TUPLE() CONSTRUCTOR

```
tuple([1, 2, 3])
```

→ (1, 2, 3)

ACCESSING & WORKING WITH TUPLES

Fast & Read-Only Operations

01. INDEXING

t = ("A", "B", "C", "D")

+ POS	0	1	2	3
VALUE	"A"	"B"	"C"	"D"
- NEG	-4	-3	-2	-1

t[0] → "A"

t[-1] → "D"



03. OPERATORS & METHODS

count() & index()

Concatenation
(+)

(1, 2) + (3, 4)

→ (1, 2, 3, 4)

Repetition
(*)

(1, 2) * 2

→ (1, 2, 1, 2)

Membership
(in)

2 in (1, 2, 3)

→ True



02. SLICING

t = (10, 20, 30, 40, 50)

t[:4]

10

20

30

40

50

→ (20, 30, 40)

t[::-1] → Returns new tuple (50, 40, 30, 20, 10)



04 ITERATION for-loop traversal

```
for item in ("A", "B", "C"):
    print(item)
```

"A"

"B"

"C"

Built-ins: len(), min(), max(), sum()

TUPLE CAPABILITIES SUMMARY

✓ ACCESS

✓ SLICE

✓ SEARCH

✓ ITERATE

✗ MODIFY

IMMUTABILITY, PATTERNS & COMPARISON

⚠

Reassignment Attempt

✖ TypeError

```
t = (10, 20, 30)
t[1] = 200 # Error!
```

TypeError: 'tuple' object does not support item assignment

del t[0] del t

✖ TypeError

✔ Delete Binding

💡

Immutability Nuance

CRITICAL

```
t = ([1, 2], 3)
t[0].append(4) # ([1, 2, 4], 3)
```

Reference is fixed, but nested mutable objects inside can still be changed.

📦 01 PACKING & UNPACKING

```
# Packing
user = "Vamshi", 22, "AI"
# Unpacking
name, age, branch = user
```

Variables match positionally.

↔ 02 SWAPPING

```
a, b = 10, 20
a, b = b, a # Tuple Swap
```

a → 20 b → 10

👤 03 NESTED TUPLES

```
data = (("A", 90), ("B", 85))
score = data[0][1]
```

[0]=Inner, [1]=Score

→ 90

List vs. Tuple Comparison

Rule: Use Tuple for fixed data | Use List when size/content changes

FEATURE	LIST []	TUPLE ()	DICTIONARY KEY HASHABILITY
Mutability	✔ Mutable	🔒 Immutable	List: Never allowed as Key
Add / Remove / Reassign	✔ Supported	✖ Not Supported	Tuple: Allowed ONLY if all elements are hashable

SETS IN PYTHON

Unordered, unique collections



Automatic Deduplication numbers = {1, 2, 3, 3, 4}

INPUT RAW DATA (CONTAINS DUPLICATE)



RESULTING SET OBJECT

{1, 2, 3, 4}

Unordered: No Positional Indexing

s[0] → **TypeError**



UNIQUE

Duplicate elements are automatically eliminated upon insertion.



UNORDERED

Elements have no fixed position, index, or slice support.



MUTABLE

Elements can be added or removed after the set is created.



HASHABLE ELEMENTS

Elements must be immutable (numbers, strings, tuples).



How a Set is Created

✓ Set

1. BASIC SYNTAX

```
s = {"apple", "banana"}
```

2. EMPTY SET SYNTAX

set() ✓ Set

{ } ✗ Dict



Mixed Types Allowed: {"Vamshi", 22, (8, 9)}

3. SET() CONSTRUCTOR

```
set([1, 2, 2, 3])
```

→ {1, 2, 3}

CREATING & MODIFYING SETS

In-Place Mutation & O(1) Search

add(x) — Single Element

SINGLE ITEM

```
s = {"apple", "banana"}  
s.add("mango")
```

→ {"apple", "banana", "mango"}



update(iterable) — Mult BULK INSERT

```
s = {"apple"}  
s.update(["kiwi", "grape"])
```

→ {"apple", "kiwi", "grape"}

remove(x)

Strict

```
s = {1, 2, 3}  
s.remove(2)
```

✗ Raises `KeyError` if absent

discard(x)

Safe

```
s = {1, 2, 3}  
s.discard(99)
```

✓ Safe: No error if missing

pop()

Arbitrary

```
s = {10, 20, 30}  
val = s.pop()
```

✗ Removes arbitrary item

clear()

Reset

```
s = {1, 2, 3}  
s.clear()
```

🗑️ Result → set()



O(1) Fast Membership Testing

HASH LOOKUP

20 in {10, 20, 30}

→ True

50 not in {10, 20}

→ True

USEFUL BUILT-IN FUNCTIONS

len(s)

min(s)

max(s)

sum(s)

SET CAPABILITIES SUMMARY

✓ ADD

✓ UPDATE

✓ REMOVE

✓ DISCARD

✓ SEARCH O(1)

✗ INDEX / SLICE

SET OPERATIONS & COMPARISON

Set Algebra & List vs Set Matrix

01 UNION (|) A.union(B)

$A \cup B$

`{1, 2} | {2, 3} → {1, 2, 3}`

All elements from both sets.

02 INTERSECTION (&) A.intersection(B)

$A \cap B$

`{1, 2} & {2, 3} → {2}`

Common elements only.

03 DIFFERENCE (-) A.difference(B)

$A - B$

`{1, 2} - {2, 3} → {1}`

Elements in A but not B.

04 SYM. DIFF (^) A.sym_diff(B)

$A \Delta B$

`{1, 2} ^ {2, 3} → {1, 3}`

Elements in either, not both.

Relationship Checks

```
{1, 2}.issubset({1, 2, 3}) → True
{1, 2}.isdisjoint({3, 4}) → True
```

Set Comprehension

```
{x*x for x in [1, 2, 2, 3]}
# Output: {1, 4, 9} (Deduplicated)
```



frozenset

IMMUTABLE

An immutable set object. Hashable and valid as a dictionary key or set element.



List vs. Set Decision Framework

Rule: Use List for ordered sequences | Use Set for uniqueness & fast membership

FEATURE	LIST []	SET {}	PRIMARY USE CASE
Ordering & Indexing	✓ Ordered (Indexable)	✗ Unordered (No Index)	List: Positional access & sequence order matter
Duplicates & Lookup	✓ Duplicates Allowed	▼ Unique Only (O(1) Search)	Set: Removing duplicates & fast membership testing

DICTIONARIES IN PYTHON

Key-Value Mapping Structure

Key (Unique & Hashable)	Mapping	Value (Any Data Type)
"name"	→	"Rahul"
"age"	→	22
"branch"	→	"AI"

Access Value by Key: student["name"]

→ "Rahul"

Key-Value Mappings & Hash Lookups

KEY → VALUE

Data is stored in associated pairs linking a unique key to a value.

UNIQUE KEYS

Keys must be unique; re-assigning an existing key updates its value.

MUTABLE

Pairs can be added, updated, or deleted after dictionary creation.

MEANINGFUL ACCESS

Values are retrieved instantly via named keys rather than numeric indices.



How a Dictionary is Created

 Key Restriction: Keys MUST be Hashable (Immutable: strings, numbers, tuples)

1. LITERAL SYNTAX

```
d = {"name": "Rahul", "age": 22}
```

2. EMPTY DICTIONARY

```
d = {} dict object
```

3. DICT() CONSTRUCTOR

```
dict(name="Rahul", age=22) → {'name': 'Rahul', ...}
```

WORKING WITH DICTIONARIES

⚡(1) Key Lookups, Reassignment & Views

🔍 01 ACCESSING & SAFE LOOKUP `d = {"name": "Rahul", "age": 22}`

Bracket Access: `d["name"]`

→ "Rahul"

Missing Key Bracket:

`d["city"]`

✗ `KeyError`

Safe `get()` Method:

`d.get("city", "Unknown")`

→ "Unknown"



02 ADDING & UPDATING

MUTABLE

NEW KEY
INSERTION

`d["city"] = "Hyderabad"`

Appends Pair

EXISTING KEY
REASSIGNMENT

`d["age"] = 23`

Replaces Value



03 REMOVING DATA

4 METHODS

`del d[k]`

`del d["age"]`

Deletes key-value pair

`pop(k)`

`val = d.pop("age")`

Removes & returns val

`popitem()`

`k, v = d.popitem()`

Removes LAST inserted

`clear()`

`d.clear()`

Empties dict → {}



04 MEMBERSHIP & ITERATION

"name" in d → True

```
for k, v in d.items():  
    print(k, "→", v)
```

Iteration Rules:

- Direct loop yields Key
- `.items()` yields (Key, Val)

📖 DICTIONARY CAPABILITIES
SUMMARY

✓ ACCESS

✓ ADD

✓ UPDATE

✓ REMOVE

✓ SEARCH KEYS O(1)

✓ ITERATE

DICTIONARY METHODS & APPLICATIONS

Dynamic Views & List vs Dict Matrix

01 keys() View d.keys()

```
{ "a": 1, "b": 2 }.keys()
→ dict_keys(['a', 'b'])
```

Dynamic view of all keys in dict.

02 values() View d.values()

```
{ "a": 1, "b": 2 }.values()
→ dict_values([1, 2])
```

Dynamic view of all values in dict.

03 items() & update() d.update(d2)

```
d.update({ "age": 23, "city": "Hyd" })
→ Bulk inserts or updates pairs
```

Merges another dictionary in-place.

Nested Dictionaries

```
users = { "s1": { "name": "Rahul", "score": 90 } }
score = users["s1"]["score"]
```

→ 90

Dictionary Comprehension

```
{x: x*x for x in [1, 2, 3]}
# Output: {1: 1, 2: 4, 3: 9}
```



List vs. Dictionary Selection Framework

FEATURE	LIST []	DICTIONARY { }	PRIMARY USE CASE
Access Method	Numeric Index lst[0]	Unique Key dict["key"]	List: Sequential positional access
Duplicates & Order	Duplicate values allowed	Unique Keys (Values may duplicate)	Dict: Labeled entity properties & fast lookups

```
A = {1, 2, 2, 3, 4}
B = {3, 4, 5}
print(A - B)
```

What will be the output?


```
A = {1, 2, 2, 3, 4}
B = {3, 4, 5}
print(A - B)
```

What will be the output?

A {1, 2, 2}

B {1, 2}

C {2, 3, 4}

D {1, 2, 3, 4, 5}

```
A = {1, 2, 2, 3, 4}
B = {3, 4, 5}
print(A - B)
```

What will be the output?

A {1, 2, 2}

B {1,
2}

C {2, 3, 4}

D {1, 2, 3, 4, 5}

Correct answer: B. {1, 2}

Duplicate elements are automatically removed from a set, so A becomes {1, 2, 3, 4}. Subtracting B removes 3 and 4, leaving {1, 2}.

```
t = (10, [20, 30], 40)
t[1].append(50)
print(t)
```

What will be the output?

```
t = (10, [20, 30], 40)
t[1].append(50)
print(t)
```

What will be the output?

A (10, [20, 30], 40)

B (10, [20, 30, 50], 40)

C Error – Tuple cannot be
modified

D (10, 20, 30, 50, 40)

```
t = (10, [20, 30], 40)
t[1].append(50)
print(t)
```

What will be the output?

A (10, [20, 30], 40)

B (10, [20, 30, 50],
40)

C Error – Tuple cannot be modified

D (10, 20, 30, 50, 40)

Correct answer: B. (10, [20, 30, 50], 40)

The tuple itself is immutable, but the list inside it is mutable. `append(50)` changes the inner list.

```
d = {"a": 10, "b": 20, "a": 30}  
print(d)
```

What will be the output?

```
d = {"a": 10, "b": 20, "a": 30}  
print(d)
```

What will be the output?

A {"a": 10, "b": 20, "a": 30}

B {"a": 30, "b": 20}

C {"a": 10, "b": 20}

D Error – Duplicate keys are not allowed

```
d = {"a": 10, "b": 20, "a": 30}  
print(d)
```

What will be the output?

A {"a": 10, "b": 20, "a": 30}

B {"a": 30, "b":
20}

C {"a": 10, "b": 20}

D Error – Duplicate keys are not allowed

Correct answer: B. {"a": 30, "b": 20}

Dictionary keys must be unique. The second "a": 30 replaces the earlier value 10.


```
a = [10, 20, 30]
b = a
b[1] = 99
print(a)
print(b)
```

**What will be the
output?**

```
a = [10, 20, 30]
b = a
b[1] = 99
print(a)
print(b)
```

What will be the output?

A [10, 20, 30]
[10, 99, 30]

B [10, 99, 30]
[10, 20, 30]

C [10, 99, 30]
[10, 99, 30]

D Error – b is a copy of a

```
a = [10, 20, 30]
b = a
b[1] = 99
print(a)
print(b)
```

What will be the output?

A

[10, 20, 30]
[10, 99, 30]

B

[10, 99, 30]
[10, 20, 30]

C

[10, 99, 30]
[10, 99, 30]

D

Error – b is a copy of
a

Correct answer: C. [10, 99, 30] / [10, 99, 30]

b = a makes both variables refer to the same list. Changing the list through b also changes what a refers to.

PYTHON DATA STRUCTURE

OPERATIONS

OPERATION	LIST[]	TUPLE()	SET{}	DICTIONARY{}
01 Access / Indexing	✓ <code>lst[0]</code>	✓ <code>t[0]</code>	✗ No Index	✓ <code>d[key]</code> (Key-based)
02 Slicing [start:stop]	✓ <code>lst[1:4]</code>	✓ <code>t[1:4]</code>	✗ Unsupported	✗ Unsupported
03 Concatenation (+)	✓ <code>[1] + [2]</code>	✓ <code>(1,) + (2,)</code>	✗ Use <code>union()</code>	✗ Use <code>update()</code>
04 Repetition (*)	✓ <code>[1, 2] * 2</code>	✓ <code>(1, 2) * 2</code>	✗ Unsupported	✗ Unsupported
05 Membership (in)	✓ <code>x in lst</code>	✓ <code>x in t</code>	✓ <code>x in s</code> (O(1))	✓ <code>key in d</code> (Checks Keys)
06 Iteration (for x in)	✓ Sequence	✓ Sequence	✓ Unique Items	✓ Keys / <code>.items()</code>
07 Length len()	✓ Item count	✓ Item count	✓ Unique count	✓ Pair count

SEQUENCE OPS

Positional indexing, slicing, concatenation (+), and repetition (*). Supported by Lists & Tuples .

COMMON OPS

Membership (in), loop iteration, and length (len()) work on **ALL four** structures.

SET-SPECIFIC

Mathematical set algebra: Union (|), Intersection (&), Difference (-), and Sym. Diff (^).

DICTIONARY-SPECIFIC

Key-based retrieval (d[k]), safe lookups (get()), and pair iteration (.items()).

DATA STRUCTURE METHODS



LIST

SEQUENCE

append(x)

Adds single item x to end

extend(iter)

Appends all elements from iterable

insert(i, x)

Inserts x at specified index i

remove(x)

Removes first matching value x

pop([i])

Removes & returns item at index

sort() / reverse()

In-place sorting & reversal

index() / count()

Positional lookup & frequency count



TUPLE

IMMUTABLE

ONLY 2 METHODS

Tuples cannot be mutated, only read-only search methods exist.

count(x)

Returns total occurrences of value x

index(x)

Returns first index position of value x

NOTE ON BUILT-INS

len(), min(), max(), sum() are Python built-in functions, NOT Tuple methods.



SET

UNIQUE

add(x) / update()

Add single element vs bulk iterable

remove(x)

Strict deletion (raises KeyError)

discard(x)

Safe deletion (no error if missing)

pop()

Removes & returns arbitrary item

union() / intersection()

Set algebra methods (or |, &)

difference() / sym_diff()

Relative & symmetric difference

issubset() / isdisjoint()

Boolean relationship checks



DICTIONARY

MAPPING

get(k, [def])

Safe lookup (returns None/default)

keys() / values()

Returns dynamic view of keys/values

items()

Dynamic view of (key, value) pairs

update(dict2)

Bulk adds or updates key-value pairs

pop(k)

Removes key & returns value

popitem()

Removes & returns last inserted pair

setdefault(k, d)

Gets value or inserts default key



METHOD ROLES SUMMARY:

List: In-place Sequence Editing Tuple: Read-Only Lookup Set: Uniqueness & Set Algebra Dict: Key-Value Mapping

LIST vs TUPLE vs SET vs

DICTIONARY

FEATURE	LIST []	TUPLE ()	SET {}	DICTIONARY {}
Syntax Example	[10, 20, 30]	(10, 20, 30)	{10, 20, 30}	{"a": 10, "b": 20}
Ordering	Ordered Sequence	Ordered Sequence	No Positional Order	Preserves Insertion Order
Mutability	✓ Mutable	✗ Immutable	✓ Mutable	✓ Mutable
Duplicates	✓ Allowed	✓ Allowed	✗ Unique Only	Keys: ✗ Unique Values: ✓
Access Method	Numeric Index lst[0]	Numeric Index t[0]	✗ No Positional Access	Unique Key d["key"]
Element Rule	Any Python object	Any Python object	Must be Hashable	Keys Hashable / Values Any

</>SYMBOL REFERENCE:

[] Index

[::] Slice

+ Concat

* Repeat

in Search

| Union

& Intersect

– Diff

^ Sym Diff

USE A LIST

When you need an **ordered collection** that changes frequently and relies on positional indexing or duplicates.

USE A TUPLE

When data should remain **fixed and read-only**, or serve as a hashable key or record structure.

USE A SET

When you need **automatic deduplication**, fast $O(1)$ membership testing, or mathematical set operations.

USE A DICTIONARY

When data represents **labeled properties** or key-value relationships requiring fast key lookup.

NESTED STRUCTURES

What is a Nested Structure?

A **nested** structure is a structure that contains another structure inside it.

In Python, lists, dictionaries, tuples and sets can be placed inside each other.

Simple Example

```
student = {  
    "name": "Malar",  
    "skills": ["Python", "SQL"]  
}
```

Dictionary containing a List



How does it look?



Dictionary
(Outer Structure)

The outermost structure.



Key
(e.g., "skills")

A key in the dictionary.



List
(Inner Structure)

The key holds another structure.



Values
(Items in the List)

The list contains multiple values.



In Short:

- One structure → contains another structure
- Helps in organizing complex real-world data

In real-world applications, data is organized in multiple levels.
Let's see how student data can be stored using nested structures.

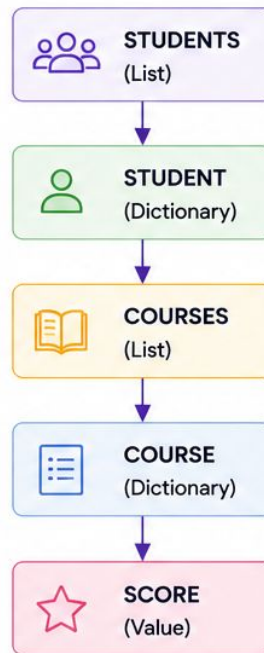
Example: Students → Courses → Scores

```
students = [  
  {  
    "name": "Arjun",  
    "courses": [  
      { "name": "Python", "score": 92 },  
      { "name": "SQL", "score": 88 }  
    ]  
  },  
  {  
    "name": "Riya",  
    "courses": [  
      { "name": "Python", "score": 85 },  
      { "name": "SQL", "score": 91 }  
    ]  
  }  
]
```

Outer most structure
List of students

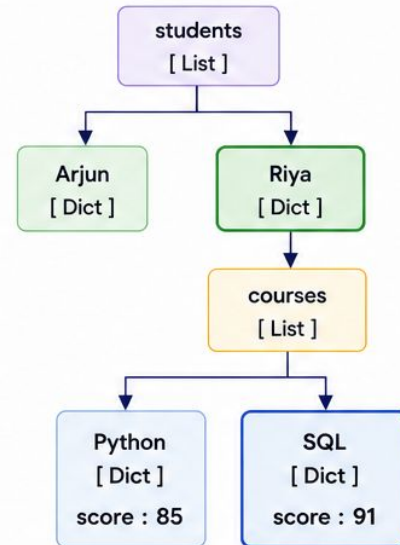
Inside each student (Dictionary),
"courses" is a List of Dictionaries.

Visual Hierarchy



 List  Dictionary  List  Dictionary  Value

Example: Riya's SQL Score



Key Takeaway: Nested structures help us represent real-world relationships in a natural and organized way.



Can you trace the data?



DATA

```
students = [  
    {  
        "name": "Arjun",  
        "courses": [  
            {"name": "Python", "score": 92},  
            {"name": "SQL", "score": 88}  
        ]  
    },  
    {  
        "name": "Riya",  
        "courses": [  
            {"name": "Python", "score": 85},  
            {"name": "SQL", "score": 91}  
        ]  
    }  
]
```



QUESTIONS

1 What will this return?

```
students[1]["courses"][0]["score"]
```

2 How would you access
Riya's SQL score?



Think carefully. Trace the path. Get the answers!



Nested Challenge: Answers Revealed!



DATA

```
students = [  
  {  
    "name": "Arjun",  
    "courses": [  
      {"name": "Python", "score": 92},  
      {"name": "SQL", "score": 88}  
    ]  
  },  
  {  
    "name": "Riya",  
    "courses": [  
      {"name": "Python", "score": 85},  
      {"name": "SQL", "score": 91}  
    ]  
  }  
]
```



ANSWERS

1 What will this return?

```
students[1]["courses"][0]["score"]
```

→

85

2 How would you access
Riya's SQL score?

```
students[1]["courses"][1]["score"]
```

→

91



Think carefully. Trace the path. Get the answers!



REAL-WORLD NESTED DATA &

JSON Patterns & Access Challenge

RECAP

</> Real-World API / JSON Record

```
company = {  
  "name": "TechCorp",  
  "employees": [  
    {"name": "Anjun", "skills": ["Python", "SQL"]},  
    {"name": "Riya", "skills": ["Java", "AWS"]}  
  ]  
}
```

Hierarchy Tree Diagram

```
company (Dict)  
├── "name": "TechCorp"  
└── "employees" (List)  
    ├── [0] (Dict Arjun)  
    │   ├── "skills" → ["Python", "SQL"]  
    │   └── [1] (Dict Riya)  
    │       └── "skills" → ["Java", "AWS"]
```

MENTAL ACCESS PIPELINE

1. Start at outer structure → 2. Identify its container type → 3. Apply [index] for List or [key] for Dict → 4. Move 1 level deeper → 5. Repeat until value reached.

COMMON PITFALLS

- Using string key on a List → **TypeError**
- Using numeric index on a Dict → **KeyError**
- Confusing order: `st[1]["courses"]` vs `st["courses"][1]`



Classroom Challenge: Evaluate Expression

`students[1]["courses"][1]["score"]`

A) 88

B) 85

✓ C) 91 (Correct)

D) "SQL"

Reasoning: `students[1]` (Riya) → `["courses"]` (List) → `[1]` (SQL Dict) → `["score"]`

→ 91



ITERATION IN PYTHON

Repeating instructions efficiently

WHAT IS ITERATION?

Repeatedly executing a block of code for multiple items in a collection or until a boolean condition turns False.

ITERATION EXECUTION FLOW

Collection → Extract Item



Perform Code Action



Move to Next Item → Repeat

```
numbers = [10, 20, 30, 40]
for n in numbers: print(n)
```

FOR LOOP

for item in iterable:

Processes each element sequentially from an iterable object.

SUPPORTED PYTHON ITERABLES

- ✓ List [1, 2]
- ✓ String "Py"
- ✓ Range range(5)
- ✓ Tuple (1, 2)
- ✓ Set {1, 2}
- ✓ Dict .items()

Note: Direct dictionary iteration yields keys by default.

WHILE LOOP

while condition:

```
i = 1
while i <= 5:
    print (i)
    i += 1 # Output: 1 2 3 4 5
```

⚠ Infinite Loop Warning: State must be updated so the condition eventually evaluates to `False`.

FOR: Known sequence

WHILE: Condition-based

ITERATION CORE FORMULA:

ITERATION

=

REPETITION

+

DATA PROCESSING

+

SEQUENTIAL ADVANCEMENT



LOOP CONTROL & DATA

MANIPULATION



01. range(start, stop, step)

Stop Excluded

```
for i in range(1, 10, 2):  
    print(i, end=" ") # Output: 1 3 5 7 9
```

Tip: range(5) yields 0, 1, 2, 3, 4

Generates numbers on demand

range(), break, continue, List & String Operations



02. break vs continue

Control Alteration

break (Exit Loop)

```
if n == 3: break  
# Output: 1, 2
```

continue (Skip Step)

```
if n == 3: continue  
# Output: 1, 2, 4, 5
```



03. LIST MANIPULATION (MUTABLE)

In-Place Mutation

append(x)

Adds to end

insert(i, x)

At index i

remove(v)

By value

pop([i])

By index

Warning: remove(value) searches by value | pop(index) removes by index.



04. STRING MANIPULATION (IMMUTABLE)

Returns New

.upper() / .lower() Case shift

.replace(o, n) Substitutes

.split(sep) Splits string into list of substrings

Immutability rule: String operations never modify original text; they return a transformed string.



QUICK MANIPULATION
SUMMARY:

+ ADD: append() / insert() - REMOVE: remove() / pop() TRANSFORM: upper() / replace()
✂ SPLIT: split()

ITERATION + MANIPULATION

Process, Filter & Transform Data



01 TRANSFORMING ELEMENTS

Index-Based Loop

```
nums = [1, 2, 3, 4, 5]
for i in range(len(nums)): nums[i] *= 2 # Output: [2, 4, 6, 8, 10]
```

Rule: Use index-based iteration `range(len())` to overwrite list values in-place.



02 FILTERING DATA

Loop + Condition + Append

```
res = []
for n in [1, 2, 3, 4, 5, 6]:
    if n % 2 == 0: res.append(n) # Output: [2, 4, 6]
```

Visual Filter: 1X|2✓|3X|4✓|5X|6✓



DICTIONARY ITERATION `.items()`

```
for k, v in student.items(): print(k, "→", v)
```

Traverses both keys and values simultaneously in a single loop.



NESTED TRAVERSAL

Multi-Level

```
for st in students: print(st["name"])
```

Traverses outer collection and accesses inner structural keys level-by-level.



Complete Data Workflow & Common Pitfalls

Iteration = WHAT to process | Manipulation = WHAT TO DO

1. COLLECTION → Input data

2. ITERATION → Visit items

3. CONDITION → Check logic

4. MANIPULATION → Transform

5. RESULT → Final output

✗ `break` exits loop (does not skip 1 item)

✗ `continue` skips 1 iteration (does not stop loop)

✗ `remove()` searches by value (not index)

✗ Strings return new objects (immutable)

```
x = [1, 2, 3]
y = x
for i in y:
    x.remove(i)
print(x)
```

What will be the output?

```
x = [1, 2, 3]
y = x
for i in y:
    x.remove(i)
print(x)
```

What will be the output?

A [1, 2, 3]

B [2, 3]

C [2]

D [3]


```
x = [1, 2, 3]
y = x
for i in y:
    x.remove(i)
print(x)
```

What will be the output?

A [1, 2, 3]

B [2, 3]

C [2]

D [3]

Correct answer: C. [2]

The loop is iterating through the same list that is being modified. After removing 1, the list becomes [2, 3], but the loop moves to the next position, which is now 3. Removing 3 leaves [2].

CONCLUSION

From storing data to processing it efficiently

Complete Course Journey: 01 to 07

Python Core Data Structures Suite

01
LIST
Ordered & mutable collections

02
TUPLE
Ordered & immutable collections

03
SETS
Unique elements & set operations

04
DICTIONARIES
Key → value mappings

05
METHODS & OPS
Work with & manipulate data

06
NESTED STRUCTURES
Build hierarchical data

07
ITERATION & MANIP.
Process & transform data

PROGRESSION STRATEGY
Choose Structure → Apply Operation → Process Data

WHAT WE LEARNED

- ✓ Different structures solve different data-storage problems efficiently.
- ✓ Methods and operations allow precise access and manipulation of data.
- ✓ Nested structures enable complex multi-level records to be represented.
- ✓ Iteration allows automated traversal and bulk transformation of collections.
- ✓ Choosing the right structure makes programs clearer, faster, and easier to maintain.

DATA PROCESSING PIPELINE

1. RAW DATA → Unstructured Input

2. DATA STRUCTURE → List / Tuple / Set / Dict

3. OPERATIONS → Access & Modify

4. PROCESSING → Iteration & Logic

5. USEFUL INFORMATION ✓